

Unidad VII

Clase 12

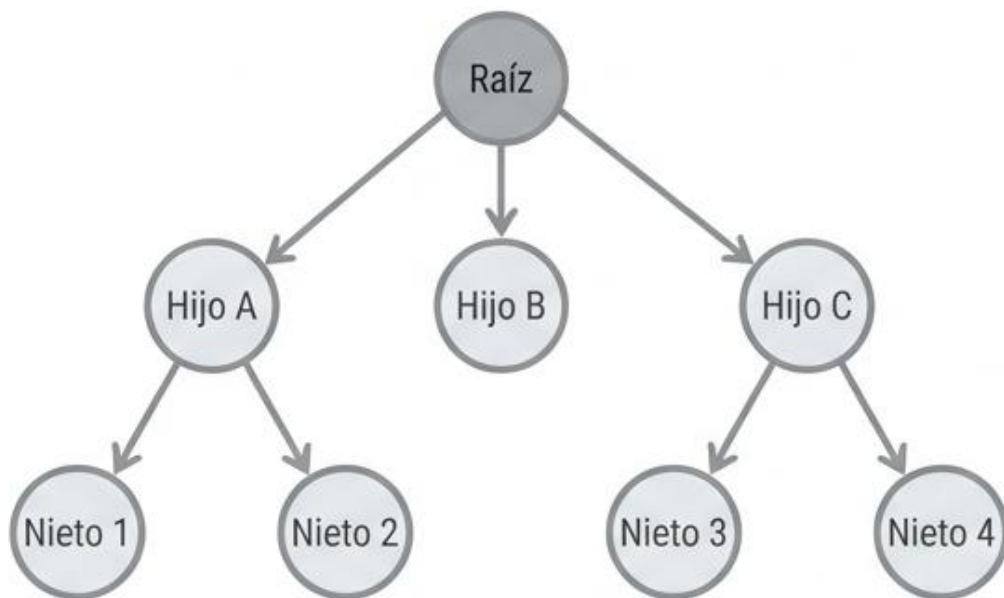
Grafos

MODELAR RELACIONES COMPLEJAS: GRAFOS

Comprender cómo modelar relaciones no jerárquicas.

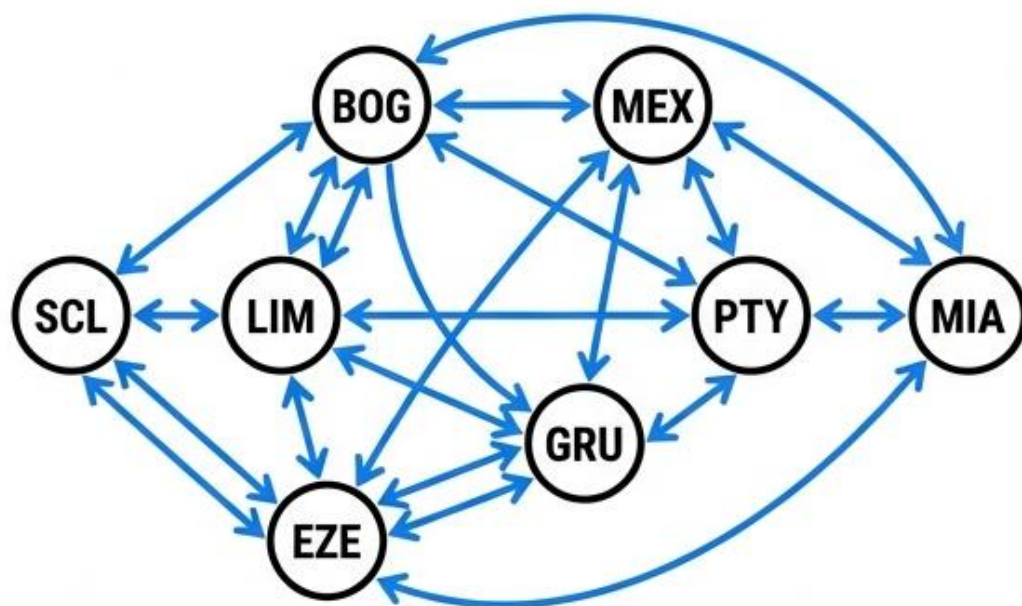
¿Por qué grafos? Modelando relaciones complejas

El Problema: Jerarquía



- Estructura estrictamente jerárquica (Padre-Hijo).
- Raíz única.
- Caminos únicamente descendentes.

La Solución: Redes



- Múltiples conexiones (circulares, cruzadas, bidireccionales).
- Ausencia de una raíz central.
- Ideal para modelar rutas, sistemas y dependencias reales.

En un grafo, la idea central no es ordenar los elementos en niveles, sino representar cómo se vinculan entre sí.

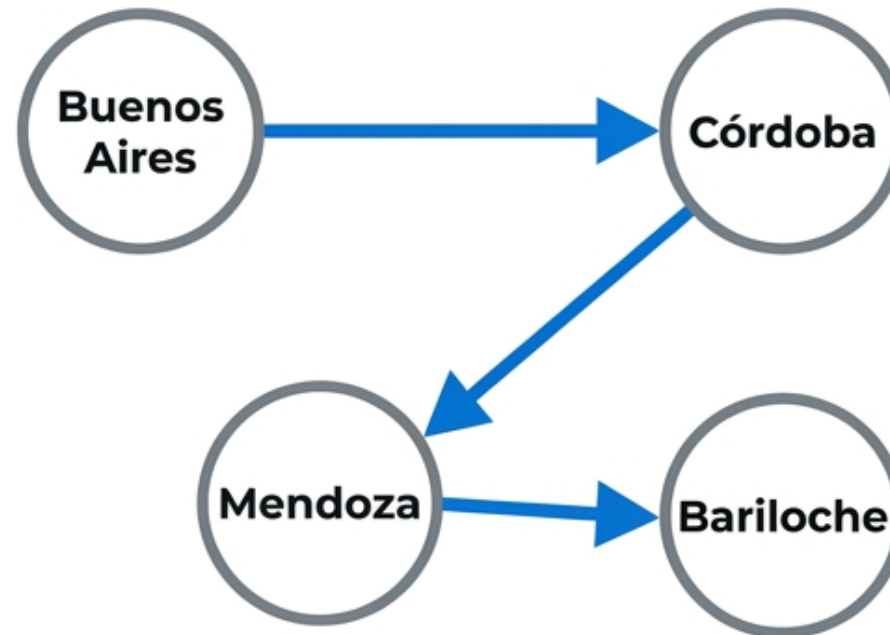
Definición formal: $G = (V, A)$

$$G = (V, A)$$

Conjunto de Vértices
(Entidades)

Conjunto de Aristas
(Relaciones directas)

$V = \{\text{Buenos Aires, Córdoba, Mendoza, Bariloche}\}$



$A = \{(\text{Buenos Aires, Córdoba}), (\text{Córdoba, Mendoza}), (\text{Mendoza, Bariloche})\}$

El grafo como modelo universal de relaciones



Logística y Transporte

V: Ciudades, Aeropuertos o Depósitos. →

A: Vuelos, Rutas terrestres o Tramos de entrega.



Redes Sociales

V: Usuarios o Perfiles. →

A: Amistades, Seguidores o Interacciones.



Sistemas Bancarios

V: Cuentas bancarias o Clientes. →

A: Transferencias o Transacciones.



Auditoría y Procesos

V: Tareas, Procesos o Módulos de software. →

A: Dependencias de ejecución o Flujo de datos.

La abstracción del modelo permite que los mismos algoritmos resuelvan problemas en disciplinas completamente distintas.

Dirección de la relación: ¿Es recíproca?

Grafo No Dirigido (Simétrico)



La relación es **bidireccional**. La existencia de (A, B) **implica automáticamente** (B, A) .

Ejemplo: Amistad recíproca en una red social.

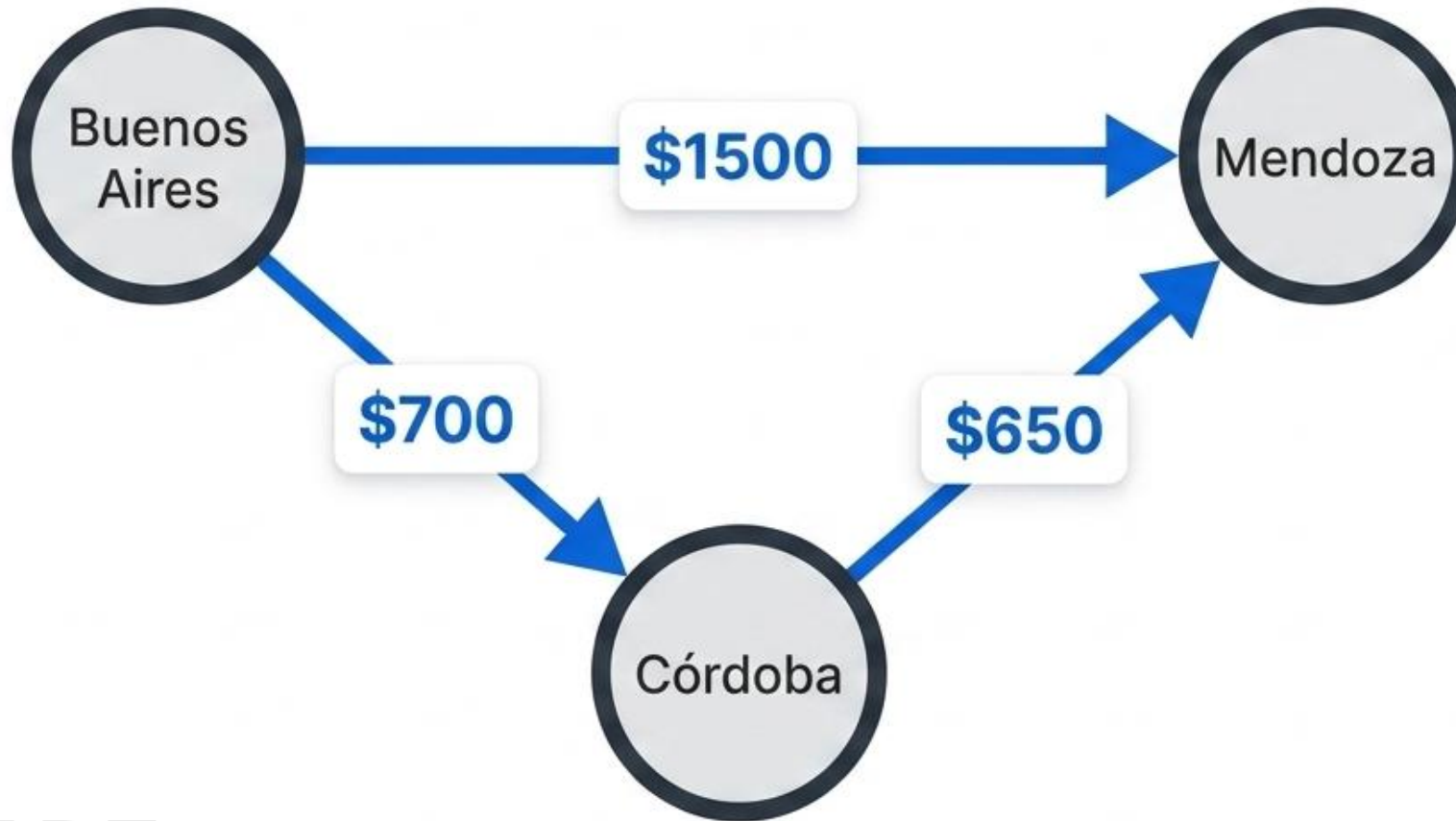
Grafo Dirigido o Digrafo (Asimétrico)



La relación tiene un **sentido estricto** (origen $\rightarrow$ destino). (A, B) y (B, A) son **aristas distintas**.

Ejemplo: Red de vuelos (Un vuelo $A \rightarrow B$ no garantiza que exista el regreso).

Grafos ponderados: El valor de la conexión



El Peso

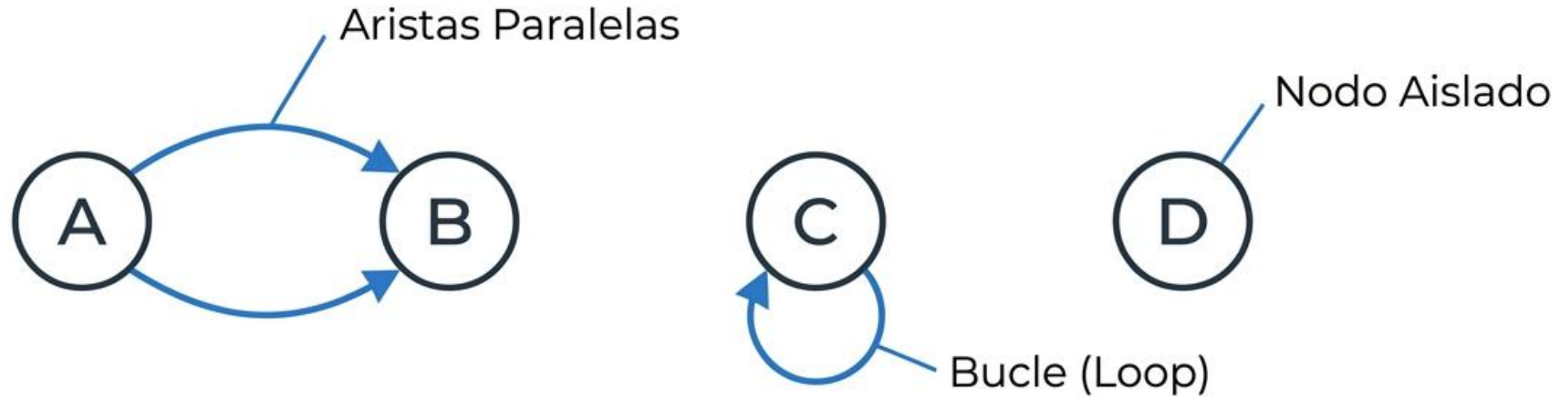
Es un valor cuantitativo asociado a la arista. Puede representar costo, distancia, tiempo, latencia o nivel de riesgo.

El Propósito

Permite responder preguntas de optimización en lugar de solo comprobar si existe conexión.

Nota: Visualmente, hacer escala en Córdoba cuesta \$1350, siendo una ruta más económica que el vuelo directo de \$1500.

Casos especiales y alcance del modelo

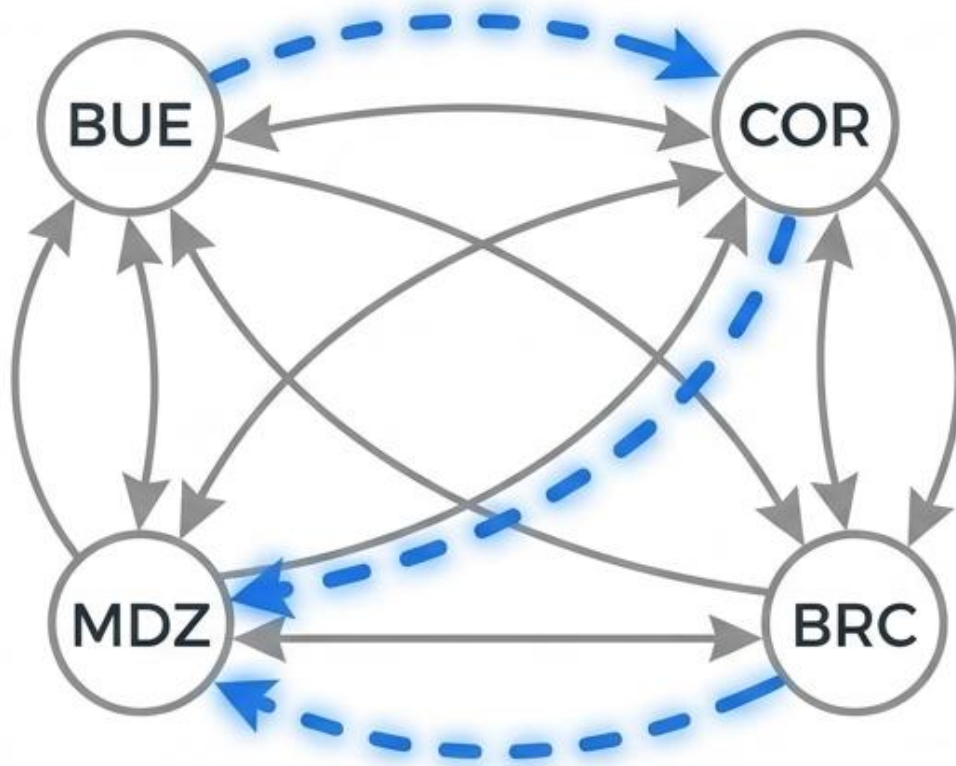


Alcance de la Unidad VII

Para simplificar el análisis inicial, en esta unidad trabajaremos exclusivamente con:

1. **Grafos dirigidos.**
2. **Sin aristas paralelas ni bucles.**
3. **Con pesos enteros positivos.**

Navegando el grafo: Caminos y Alcanzabilidad



Adyacencia y Grado: Conexión directa.
Grado Positivo = aristas salientes (ej: BUE tiene 3).
Grado Negativo = aristas entrantes.

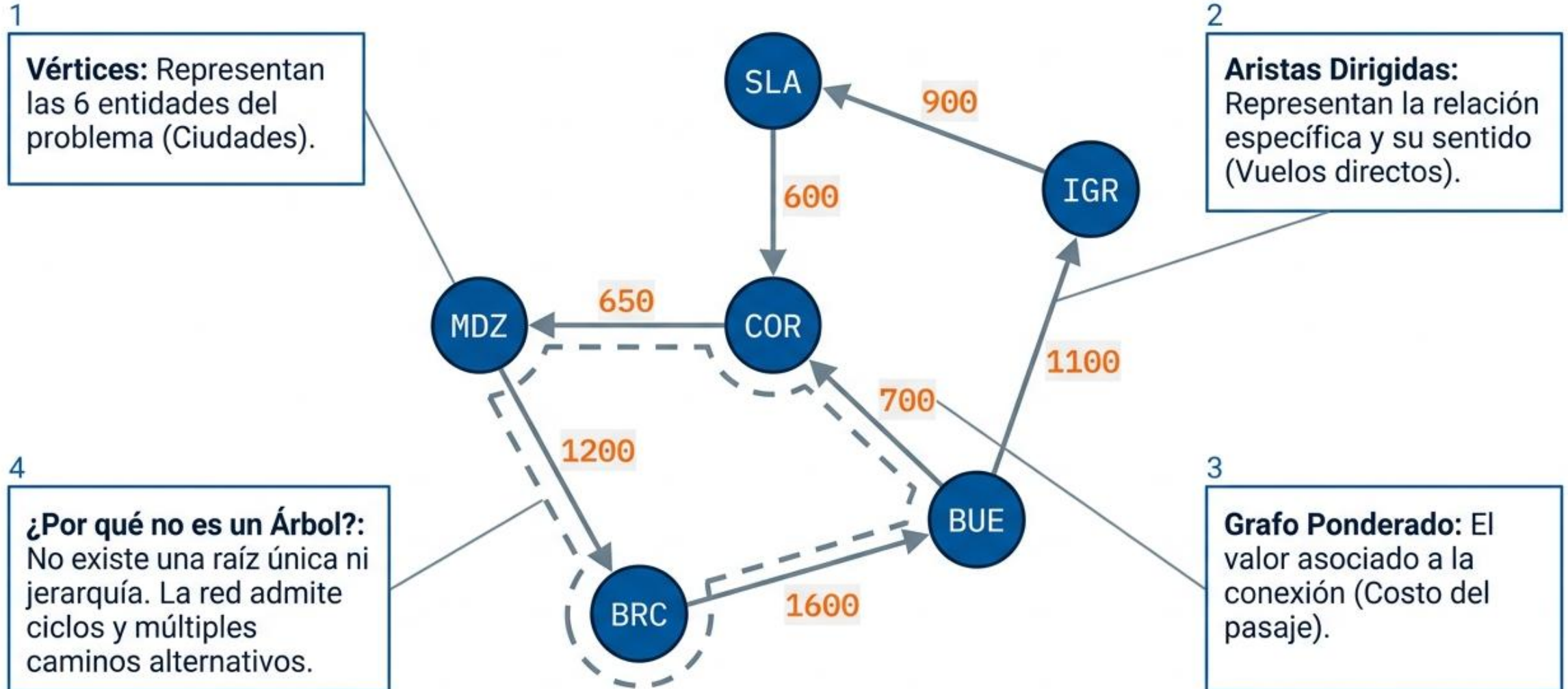
Camino: Sucesión de vértices conectados. El costo total es la suma de sus pesos.

Ciclo: Un camino que vuelve a su vértice de origen. (Requiere control algorítmico para evitar bucles infinitos).

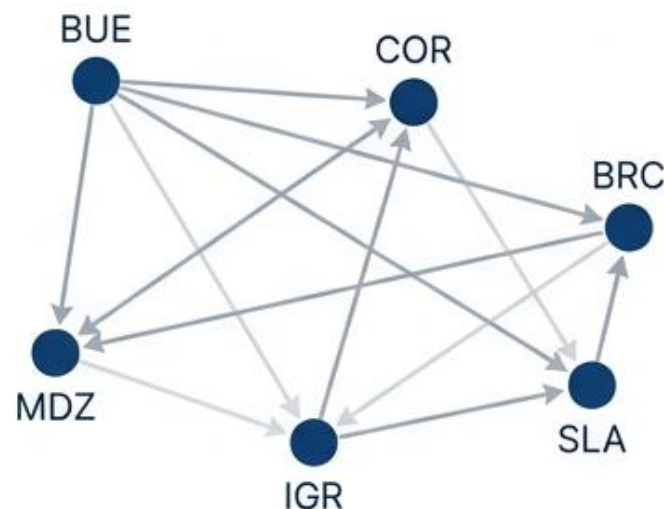
Alcanzabilidad

¿Existe al menos un camino indirecto entre A y B? Responde a la pregunta más crítica: "¿Puedo llegar desde este punto a este otro, aunque no exista una conexión directa?"

Modelando un Caso Real: Red de Vuelos Regionales



La Lente Espacial: Matriz de Adyacencia



- **Concepto:** Una matriz cuadrada de $N \times N$.
- **Mecánica:** Cada celda (i, j) guarda el peso de la arista.
- **El Vacío:** Si no existe conexión (ej. no hay vuelo $COR \rightarrow BUE$), se registra 0 o NULL.

		Destino					
		BUE	COR	MDZ	BRC	IGR	SLA
Origen	BUE	0	700	980	0	1100	0
	COR	0	0	650	0	0	0
	MDZ	0	0	0	1200	0	0
	BRC	1600	0	0	0	0	0
	IGR	0	0	0	0	0	900
	SLA	0	600	0	0	0	0

Lectura Instantánea: Consultas Directas

Origen	Destino					
	BUE	COR	MDZ	BRC	IGR	SLA
	BUE	0	700	980	0	1100
	COR	0	0	650	0	0
	MDZ	0	0	0	1200	0
	BRC	1600	0	0	0	0
	IGR	0	0	0	0	900
	SLA	0	600	0	0	0

QUERY: ¿Existe vuelo BUE -> MDZ?

RESULT: Sí, la celda indica **980**.

QUERY: ¿Existe vuelo COR -> BUE?

RESULT: No, la celda indica **0**.

QUERY: ¿Cuál es el costo de MDZ -> BRC?

RESULT: El costo es **1200**.

Acceso Directo: La matriz responde consultas entre cualquier par de vértices de forma directa e instantánea consultando una sola celda.

El Costo de la Velocidad: Ventajas y Límites

La Ventaja (Velocidad)

200	450	150	300
100	600	250	500
120	700	350	400
180	550	220	480

Favorece consultas ultrarrápidas de existencia de aristas. Es la estructura ideal para Grafos Densos (donde casi todos los nodos están interconectados).

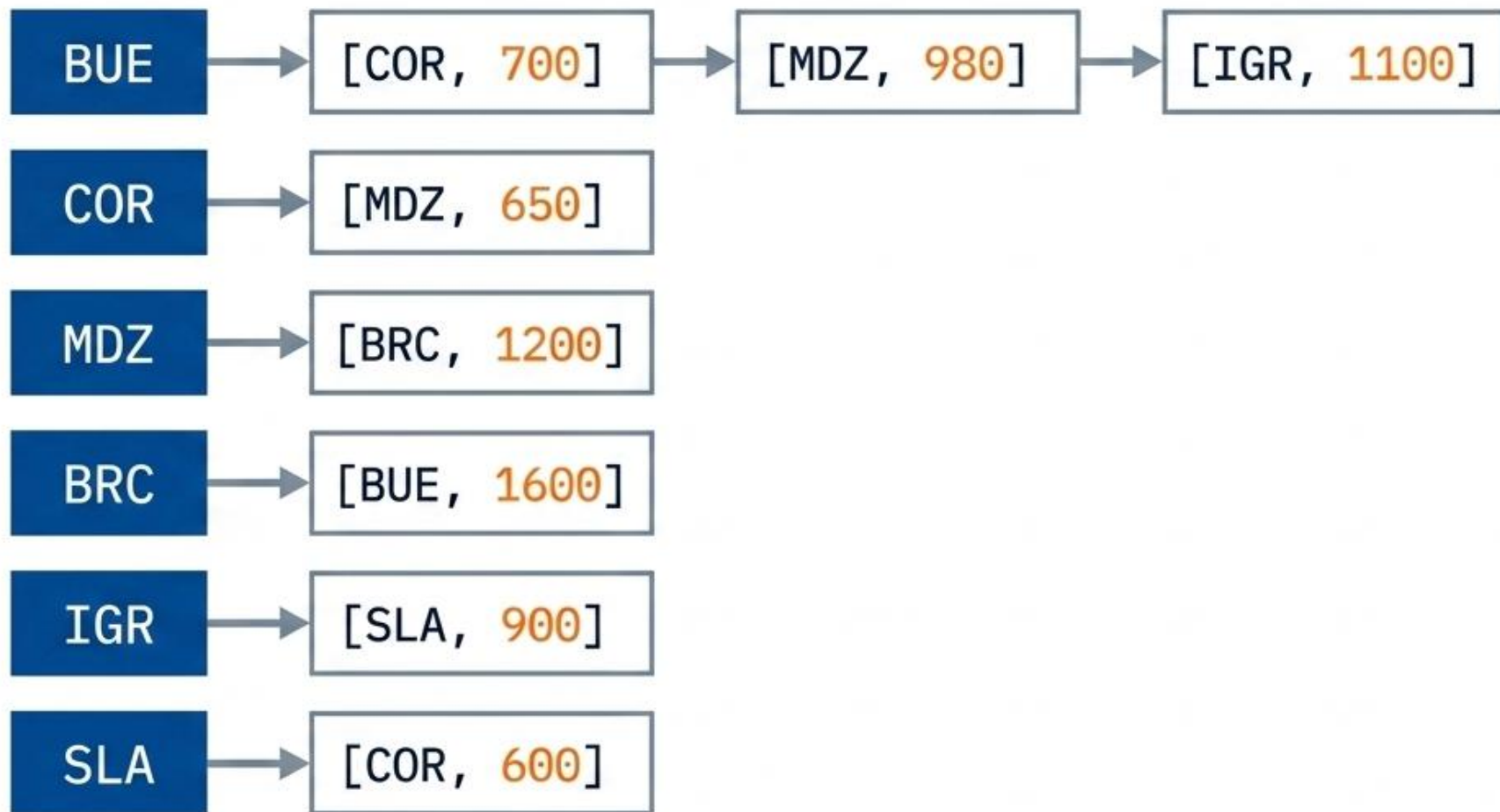
El Límite (Memoria Desperdiciada)



Consume memoria $N \times N$ sin importar cuántas conexiones reales existan.

Ejemplo de Escala: 1,000 ciudades implican 1,000,000 de celdas. Si solo hay 3,000 vuelos, el 99.7% de la memoria almacenará 'ausencia de vuelo'.

La Lente Relacional: Lista de Adyacencia

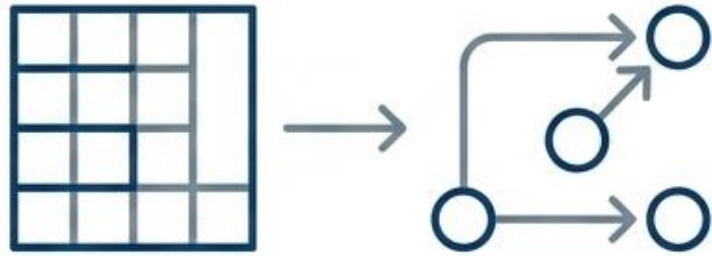


Concepto: Cada vértice guarda únicamente un arreglo o lista de sus destinos directos.

Eficiencia: En grafos ponderados, el nodo destino incluye su peso. Lo que no existe, simplemente no ocupa espacio.

Eficiencia Espacial: Ventajas y Límites

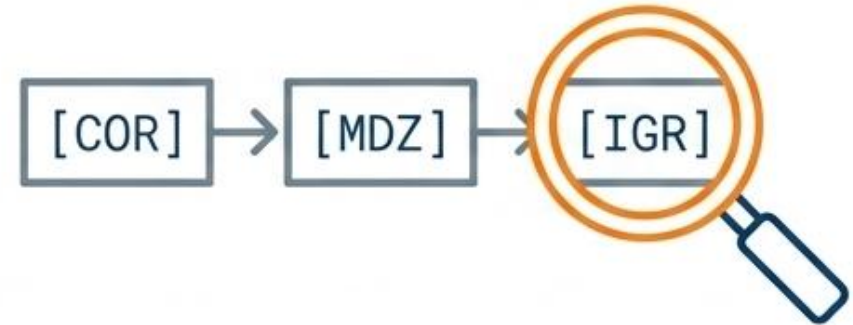
La Ventaja (Memoria y Recorrido)



Ahorro masivo de memoria en Grafos Dispersos (muchos nodos, pocas conexiones).

Resulta la estructura más natural para listar vecinos y recorrer el grafo para buscar rutas (algoritmos BFS/DFS).

El Límite (Tiempo de Consulta)



Consultar la existencia de una arista puntual es más lento.

Ejemplo: Para saber si BUE vuela a IGR, el sistema no puede mirar una celda directa; debe recorrer la lista de BUE secuencialmente hasta encontrar IGR o llegar al final.

Criterios de Decisión: El Veredicto

The Blueprint / Minimalist Technical Identity

Criterio	Matriz de Adyacencia	Lista de Adyacencia
Memoria	N x N (Inflexible)	Proporcional a las aristas (Eficiente)
Consulta Directa (A -> B)	Instantánea / Directa	Secuencial (Requiere recorrer)
Recorrer Vecinos / Explorar	Lento (Debe revisar ceros)	Muy rápido y natural
Tipo de Grafo Ideal	Grafos Densos	Grafos Dispersos

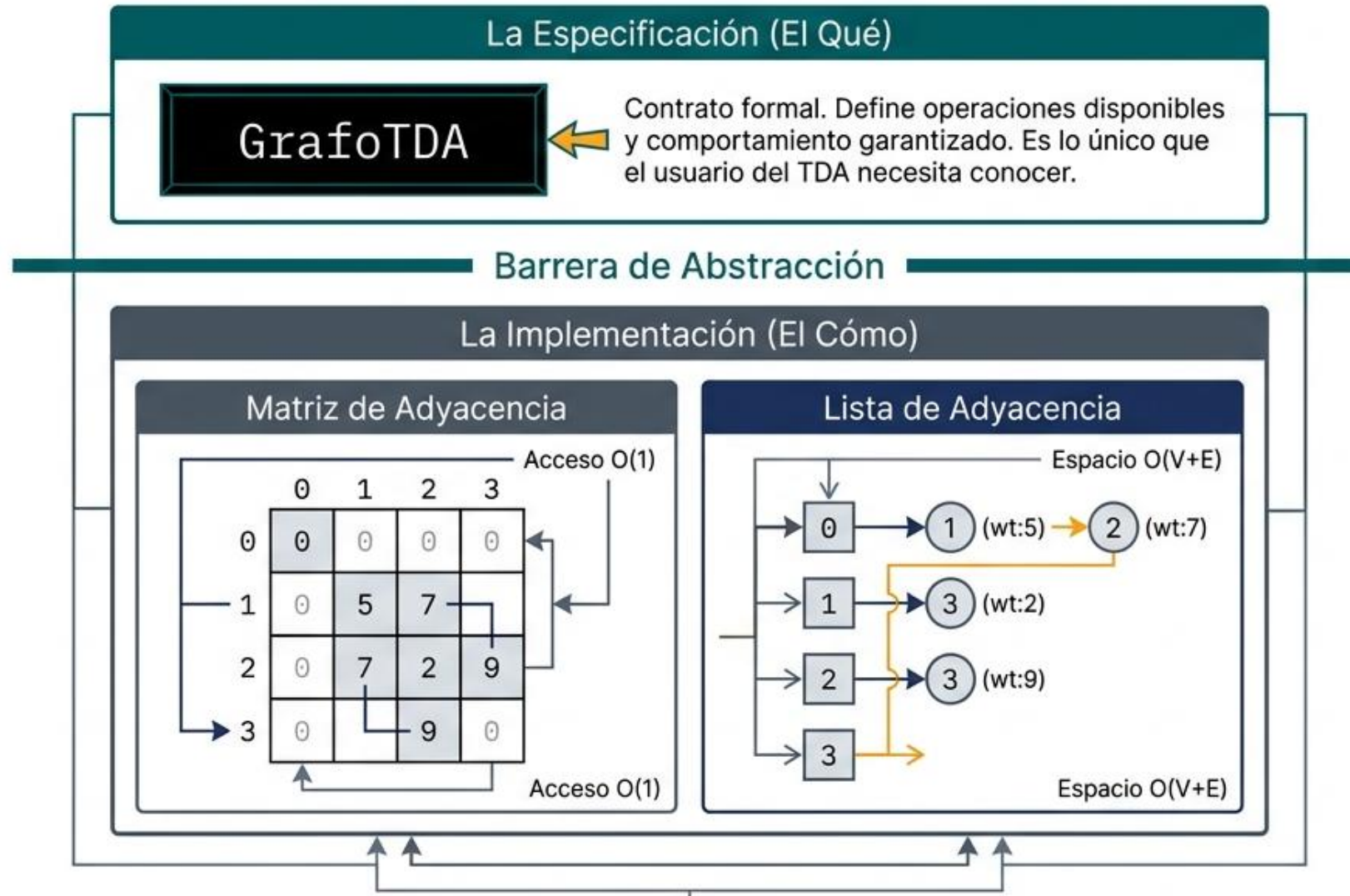
Regla Práctica

Utilizá la Matriz si el grafo es denso o tu prioridad absoluta es consultar conexiones específicas entre pares.

Utilizá la **Lista** si el grafo es disperso (la mayoría de las redes reales) y te interesa ahorrar memoria o recorrer los vecinos.

La Barrera de Abstracción: Especificación vs. Implementación

Un TDA define el comportamiento esperado sin imponer la estructura interna.



El código que usa el TDA no cambia, sin importar cuál de estos 'motores' esté funcionando por debajo.

El Panel de Control: Operaciones Básicas del GrafoTDA

Agrupación lógica de la interfaz Java para interactuar con la estructura.



Construcción

Preparan la estructura y agregan los elementos base.

```
void InicializarGrafo();  
void AgregarVertice(int v);  
void AgregarArista(int v1,  
int v2, int peso);
```



Modificación

Alteran el estado interno eliminando elementos.

```
void EliminarVertice(int  
v);  
void EliminarArista(int v1,  
int v2);
```



Consulta

Extraen información sin modificar la estructura.

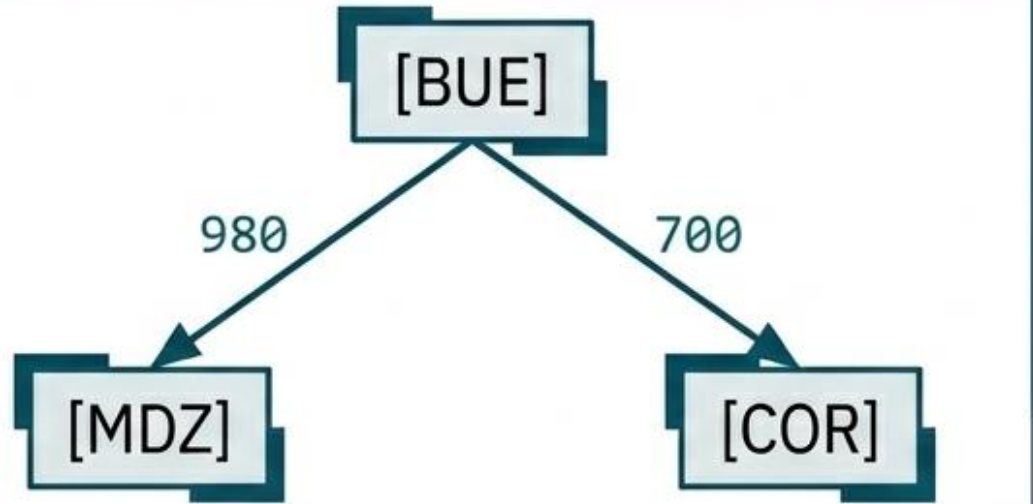
```
ConjuntoTDA Vertices();  
boolean ExisteArista(int  
v1, int v2);  
int PesoArista(int v1, int  
v2);
```

Sobre estas operaciones primitivas construiremos comportamientos más complejos, como obtener adyacentes o recorrer caminos.

Uso Programático: Interactuando con la Caja Negra

Consultas directas utilizando únicamente los métodos de la interfaz.

El Modelo



La Interacción

```
g.ExisteArista(BUE, MDZ)  
true
```

🔍 ¿Hay vuelo directo? Sí

```
g.PesoArista(BUE, MDZ)  
980
```

🏷️ ¿Cuál es el costo directo?

⚠ El límite de la consulta primitiva

¿Puedo llegar de Buenos Aires a Bariloche con escalas?

Las operaciones del TDA responden preguntas de un solo paso. Responder sobre conectividad indirecta o caminos requiere programar Algoritmos de Recorrido.

Construyendo Herramientas: Calcular Vecindario

Un algoritmo que utiliza el TDA como bloque de construcción sin acceder a sus datos internos.

Code Walkthrough

```
public ConjuntoTDA vecindario(GrafoTDA g, ConjuntoTDA base) {  
    ConjuntoTDA vecinos = new ConjuntoLD();  
    vecinos.InicializarConjunto();  
    ConjuntoTDA copiaBase = copiarConjunto(base);  
  
    while (!copiaBase.ConjuntoVacio()) {  
        int v = copiaBase.Elegir();  
        copiaBase.Sacar(v);  
        ConjuntoTDA todos = g.Vertices();  
  
        while (!todos.ConjuntoVacio()) {  
            int w = todos.Elegir();  
            todos.Sacar(w);  
            if (g.ExisteArista(v, w)) {  
                vecinos.Agregar(w);  
            }  
        }  
    }  
    return vecinos;  
}
```

El Universo. Obtenemos todos los vértices del grafo mediante el TDA.

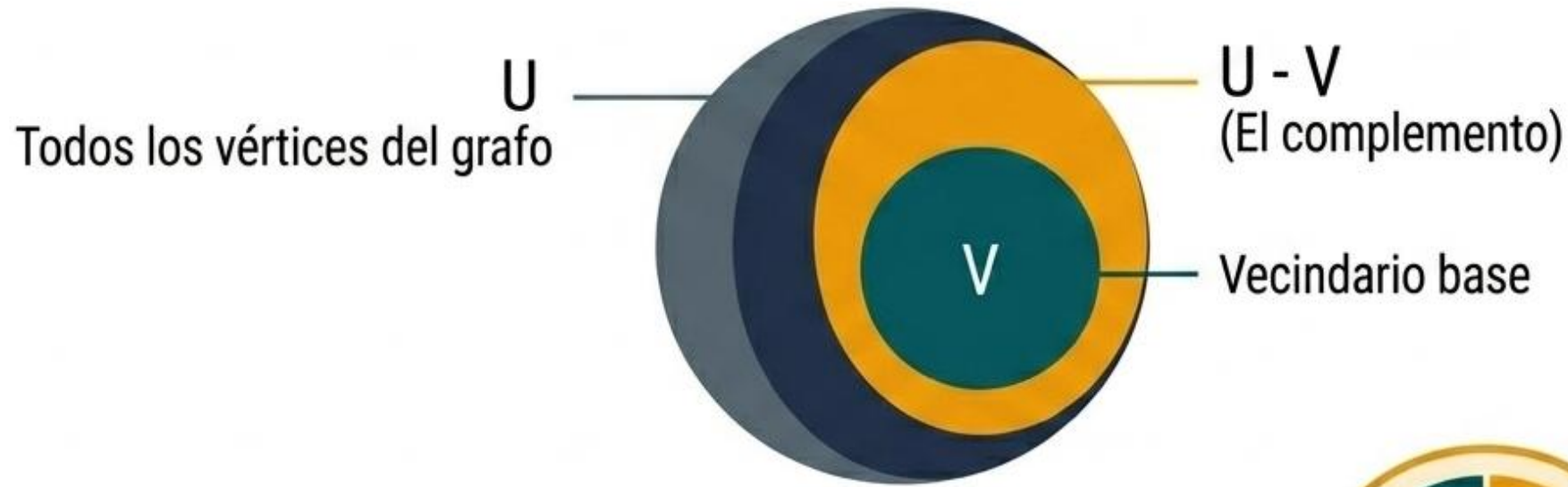
El Puente Lógico. Delegamos en el TDA la responsabilidad de saber si están conectados.



100% Modularidad. No hay matrices ni punteros visibles. Este código funciona perfecto ya sea que el Grafo use Matriz o Lista.

Expandiendo la Lógica: Complemento del Vecindario

Combinando operaciones de conjuntos con algoritmos de grafos.



La Lógica

- [1] Calcular **vecindario** del conjunto base.
- [2] Obtener **Vertices()** totales del grafo.
- [3] Iterar sobre los vecinos calculados y extraerlos (**Sacar()**) del conjunto total.



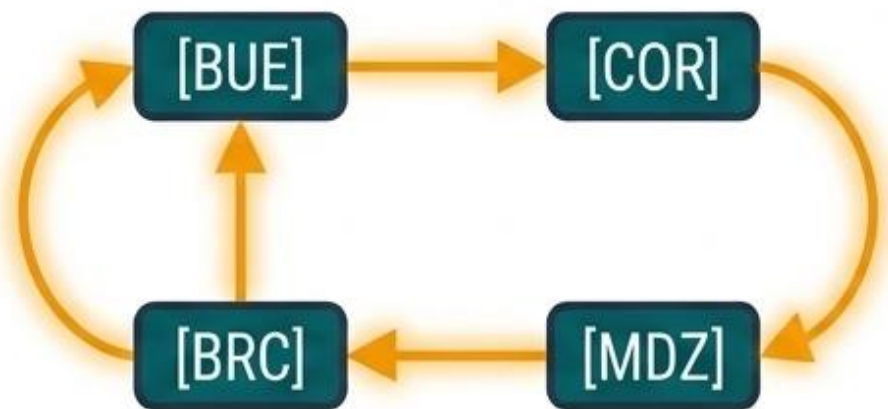
Costo Teórico: $O(n^2)$ en el peor caso (donde n = vértices).

Explicación: El algoritmo compara iterativamente cada vértice del conjunto base contra todos los vértices del grafo. El costo final dependerá del tamaño del conjunto base y de la complejidad interna de las operaciones de ConjuntoTDA.

El Problema de los Ciclos y la Marca de Visitados

Recorrer un grafo no es como recorrer un árbol: la falta de jerarquía estricta exige control de memoria.

(The Danger of Circularity)



(The Solution: "El Pasaporte")



Árboles (Estructura jerárquica)

Siempre avanzan hacia abajo (raíz a hojas). No hay riesgo de bucles infinitos por su definición estructural.



Grafos (Red interconectada)

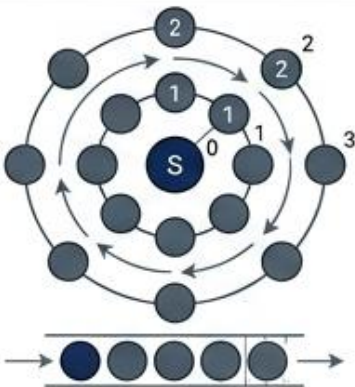
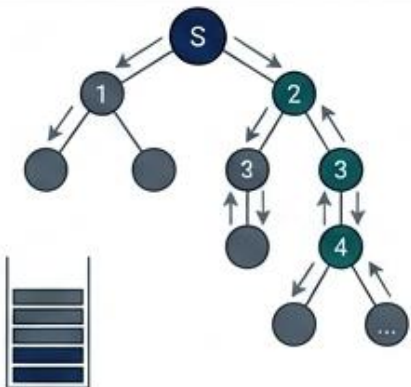
Pueden tener ciclos. Sin un registro estricto de vértices visitados (un conjunto auxiliar o array de marcas), un algoritmo de exploración quedará atrapado en un recorrido infinito repitiendo el ciclo geométrico.



Algoritmos de Exploración y Síntesis Estructural

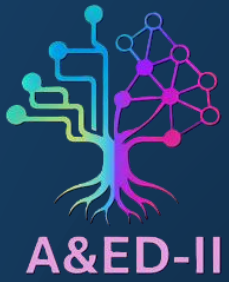
Las herramientas maestras de recorrido y la arquitectura de la solución completa.

Comparativa de Exploración

BFS (Búsqueda en Anchura)		DFS (Búsqueda en Profundidad)	
Mecanismo: Usa estructura Cola (FIFO). Avanza por niveles/distancia concéntrica. Uso: Ideal para encontrar el camino más corto o menor cantidad de saltos.		Mecanismo: Usa Pila o Recursividad (LIFO). Avanza hasta el fondo de un camino antes de retroceder (Backtracking). Uso: Ideal para exploración completa, encontrar ciclos y analizar dependencias profundas.	

La Síntesis: Embudo de 3 Niveles

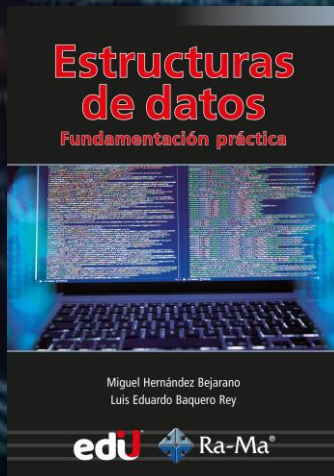
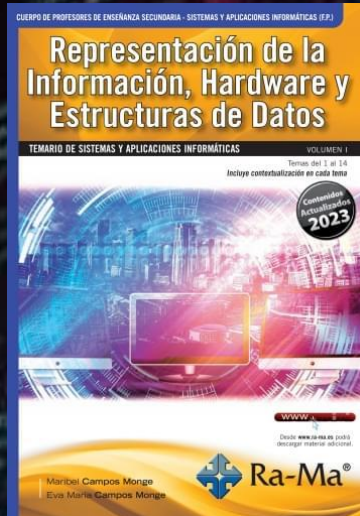




Actividad

La empresa necesita representar esta red mediante un grafo para poder consultar conexiones directas, analizar rutas posibles, identificar vecindarios de ciudades y justificar qué estructura conviene usar para almacenar la información.

Una empresa regional de transporte y logística opera una red de conexiones entre ciudades. Cada ciudad puede tener rutas directas hacia otras ciudades, pero las conexiones no siempre son recíprocas: puede existir una ruta desde una ciudad A hacia una ciudad B, sin que exista necesariamente una ruta directa de regreso. Además, cada conexión tiene un costo operativo estimado.



BIBLIOGRAFIA



Cuadrado Estrebou, María Fernanda Trutner, Guillermo Hernán. Revisión: J.Tempio.
Programación II. Apuntes de cátedra.



PROXIMO ENCUENTRO



Clase 12

Modelar relaciones complejas: Grafos

Definición de grafo.

Matriz de adyacencia.

Lista de adyacencia.

Especificación del TDA.